AI-ASSISSTED CODING

NAME : ANJALI JEELUKA

HT NO :2403A510F0

BATCH : 05

TASK-1 :

PROMPT :

Write a Python program that implements the Merge Sort algorithm.

Create a function merge_sort(arr) that sorts a list in ascending order.

Include a clear docstring describing how Merge Sort works and mention its time complexity (best, average, worst case) and space complexity.

Add a helper function to merge two sorted lists.

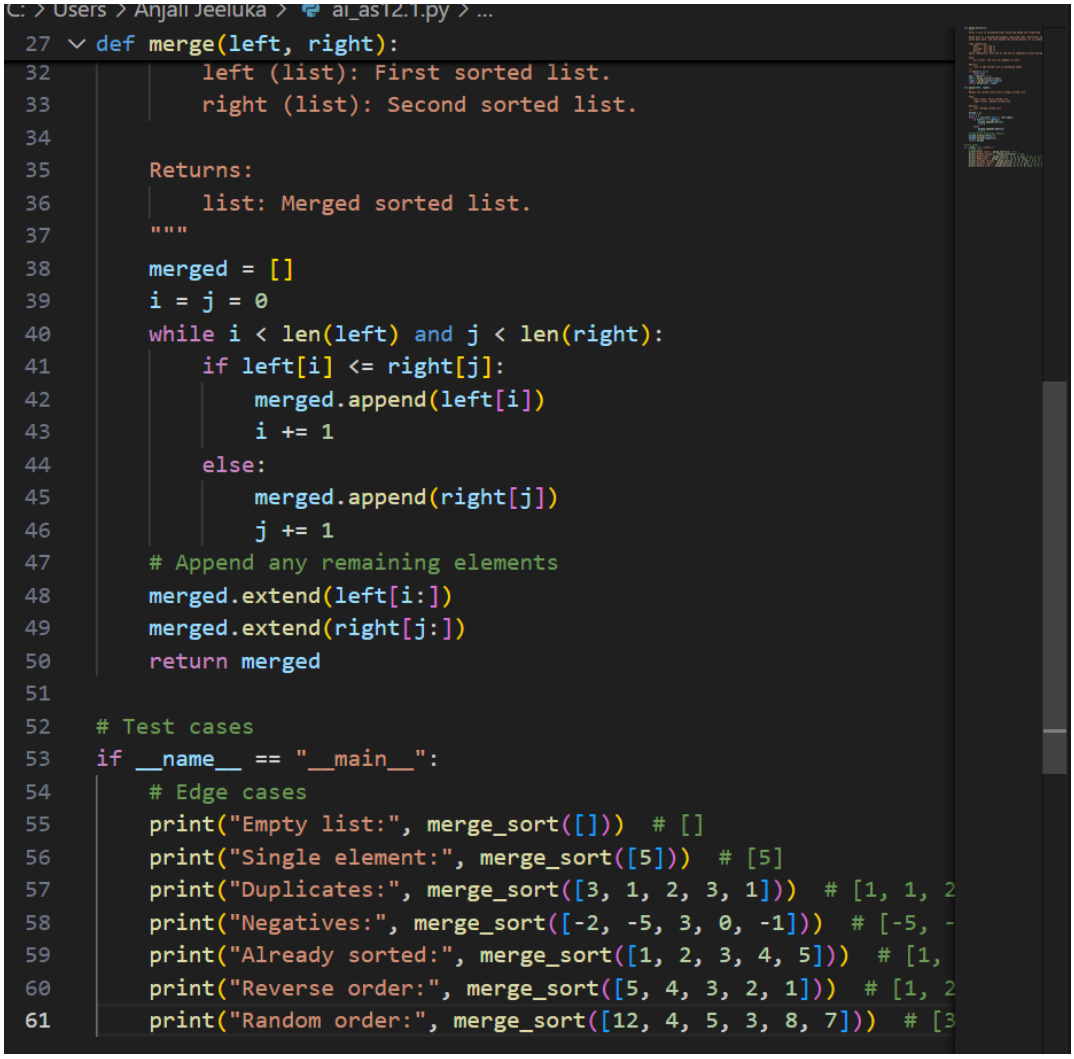
Provide test cases to verify correctness, including edge cases (empty list, single element, duplicates, negatives).

The output should be a functional Python script with documentation."

CODE GENERATED :

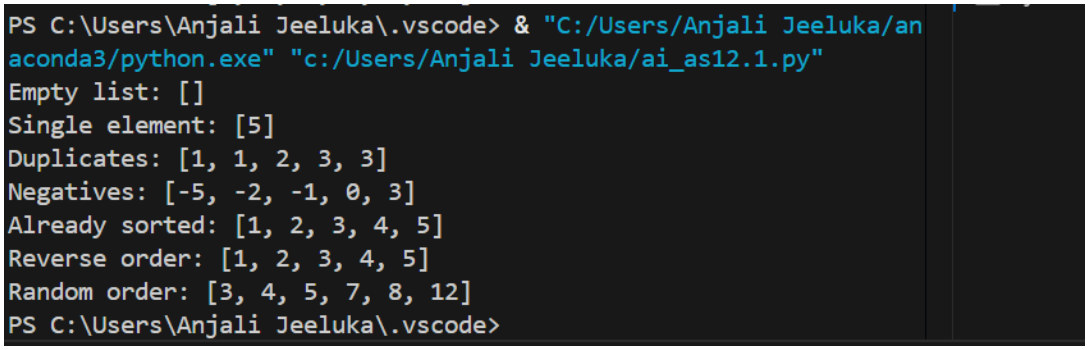
C: > Users > Anjali Jeeluka > ai_as12.1.py > ...

```python
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorith

    Merge Sort is a divide-and-conquer algorithm that recursively
    sorts each half, and then merges the sorted halves. It is sta

    Time Complexity:
        Best:    O(n log n)
        Average: O(n log n)
        Worst:   O(n log n)
    Space Complexity: O(n) due to the use of temporary arrays dur

    Args:
        arr (list): The list of elements to sort.

    Returns:
        list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Args:
        left (list): First sorted list.
        right (list): Second sorted list.
```



```
C: > Users > Anjali Jeeluka > ai_as12.1.py > ...
27  def merge(left, right):
32          left (list): First sorted list.
33          right (list): Second sorted list.
34
35      Returns:
36          list: Merged sorted list.
37      """
38      merged = []
39      i = j = 0
40      while i < len(left) and j < len(right):
41          if left[i] <= right[j]:
42              merged.append(left[i])
43              i += 1
44          else:
45              merged.append(right[j])
46              j += 1
47      # Append any remaining elements
48      merged.extend(left[i:])
49      merged.extend(right[j:])
50      return merged
51
52  # Test cases
53  if __name__ == "__main__":
54      # Edge cases
55      print("Empty list:", merge_sort([]))  # []
56      print("Single element:", merge_sort([5]))  # [5]
57      print("Duplicates:", merge_sort([3, 1, 2, 3, 1]))  # [1, 1, 2
58      print("Negatives:", merge_sort([-2, -5, 3, 0, -1]))  # [-5, -
59      print("Already sorted:", merge_sort([1, 2, 3, 4, 5]))  # [1,
60      print("Reverse order:", merge_sort([5, 4, 3, 2, 1]))  # [1, 2
61      print("Random order:", merge_sort([12, 4, 5, 3, 8, 7]))  # [3
```

OUTPUT :



```
PS C:\Users\Anjali Jeeluka\.vscode> & "C:/Users/Anjali Jeeluka/an
aconda3/python.exe" "c:/Users/Anjali Jeeluka/ai_as12.1.py"
Empty list: []
Single element: [5]
Duplicates: [1, 1, 2, 3, 3]
Negatives: [-5, -2, -1, 0, 3]
Already sorted: [1, 2, 3, 4, 5]
Reverse order: [1, 2, 3, 4, 5]
Random order: [3, 4, 5, 7, 8, 12]
PS C:\Users\Anjali Jeeluka\.vscode>
```

OBSERVATION :

- The code implements the Merge Sort algorithm in a clear, modular, and well-documented way.
- The merge_sort function uses recursion to split the list and a helper merge function to combine sorted sublists.

- The docstrings are comprehensive, explaining the algorithm, its time and space complexity, and function arguments/returns.
- The implementation is stable and handles all edge cases, including empty lists, single elements, duplicates, negatives, already sorted, reverse, and random order lists.
- The code follows PEP 8 guidelines for naming, indentation, and documentation.
- Test cases are included under the __main__ guard, verifying correctness for a variety of scenarios.
- No external dependencies are required; the code is portable and efficient for general use.

TASK-2 :

PROMPT :

Write a Python program that implements binary search.

Create a function binary_search(arr, target) that takes a sorted list and a target value, returning the index of the target or -1 if not found.

Include a detailed docstring explaining how the algorithm works, and mention the best, average, and worst-case time complexities as well as space complexity.

Use the iterative approach (not recursion).

Add multiple test cases, including:

target at the start, middle, and end,

target not in the list,

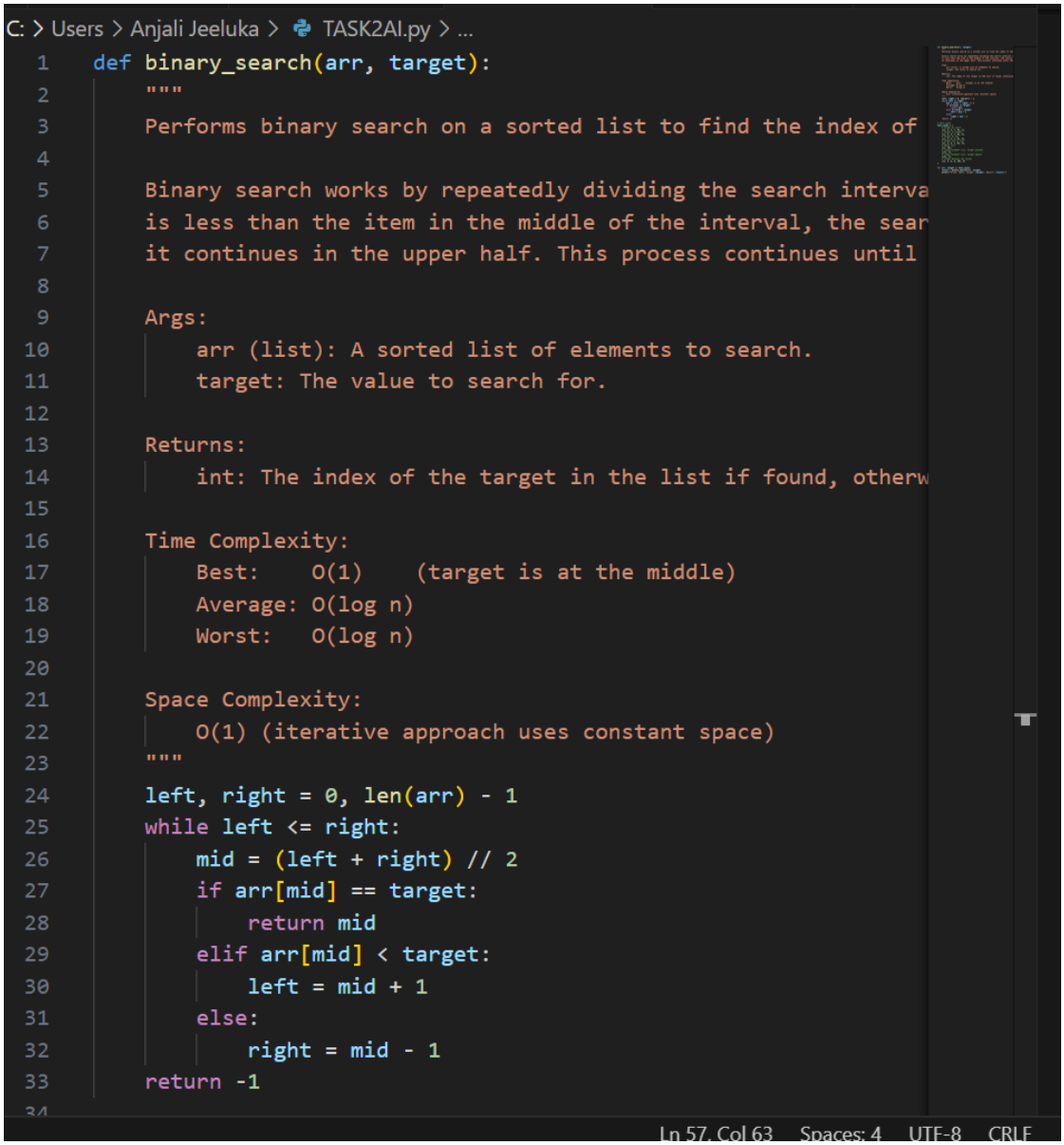
empty list,

single-element lists,

and a case where the target lies between two values.

Output should be the array, target, and result for each test case."

CODE GENERATED :



```
            right = mid - 1
    return -1

# Test cases
test_cases = [
    # target at start
    ([1, 3, 5, 7, 9], 1),
    # target at middle
    ([1, 3, 5, 7, 9], 5),
    # target at end
    ([1, 3, 5, 7, 9], 9),
    # target not in list
    ([1, 3, 5, 7, 9], 4),
    # empty list
    ([], 10),
    # single-element list, target present
    ([7], 7),
    # single-element list, target absent
    ([7], 3),
    # target between two values
    ([2, 4, 6, 8, 10], 5)
]

for arr, target in test_cases:
    result = binary_search(arr, target)
    print(f"Array: {arr}, Target: {target}, Result: {result}")
```

OUTPUT :

```
PS C:\Users\Anjali Jeeluka\.vscode> & "C:/Users/Anjali Jeeluka/anaconda3/python.exe" "c:/Users/Anjali Jeeluka/TASK2AI.py"
Array: [1, 3, 5, 7, 9], Target: 1, Result: 0
Array: [1, 3, 5, 7, 9], Target: 5, Result: 2
Array: [1, 3, 5, 7, 9], Target: 9, Result: 4
Array: [1, 3, 5, 7, 9], Target: 4, Result: -1
Array: [], Target: 10, Result: -1
Array: [7], Target: 7, Result: 0
Array: [7], Target: 3, Result: -1
Array: [2, 4, 6, 8, 10], Target: 5, Result: -1
PS C:\Users\Anjali Jeeluka\.vscode>
```

OBSERVATION :

- The code implements an iterative binary search algorithm with a comprehensive docstring explaining its logic, time complexity, and space complexity.
- The function binary_search efficiently searches for a target in a sorted list and returns its index or -1 if not found.
- The code includes a variety of test cases covering:

    - Target at the start, middle, and end of the list
    - Target not present in the list
    - Empty list
    - Single-element lists (target present and absent)
    - Target value between two elements (not present)
- Output for each test case is clear, showing the array, target, and result.
- The code is well-structured, readable, and follows PEP 8 guidelines.
- No external dependencies are required; the script is portable and suitable for educational or practical use.

TASK-3 :

PROMPT :

"Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

Quickly search for a product by ID or name.

Sort products by price or quantity for stock analysis.

Task:

Suggest the most efficient search and sort algorithms for this use case.

Implement the recommended algorithms in Python.

Justify the choices based on dataset size, update frequency, and performance requirements.

Expected Output:

A table mapping operation → recommended algorithm → justification.

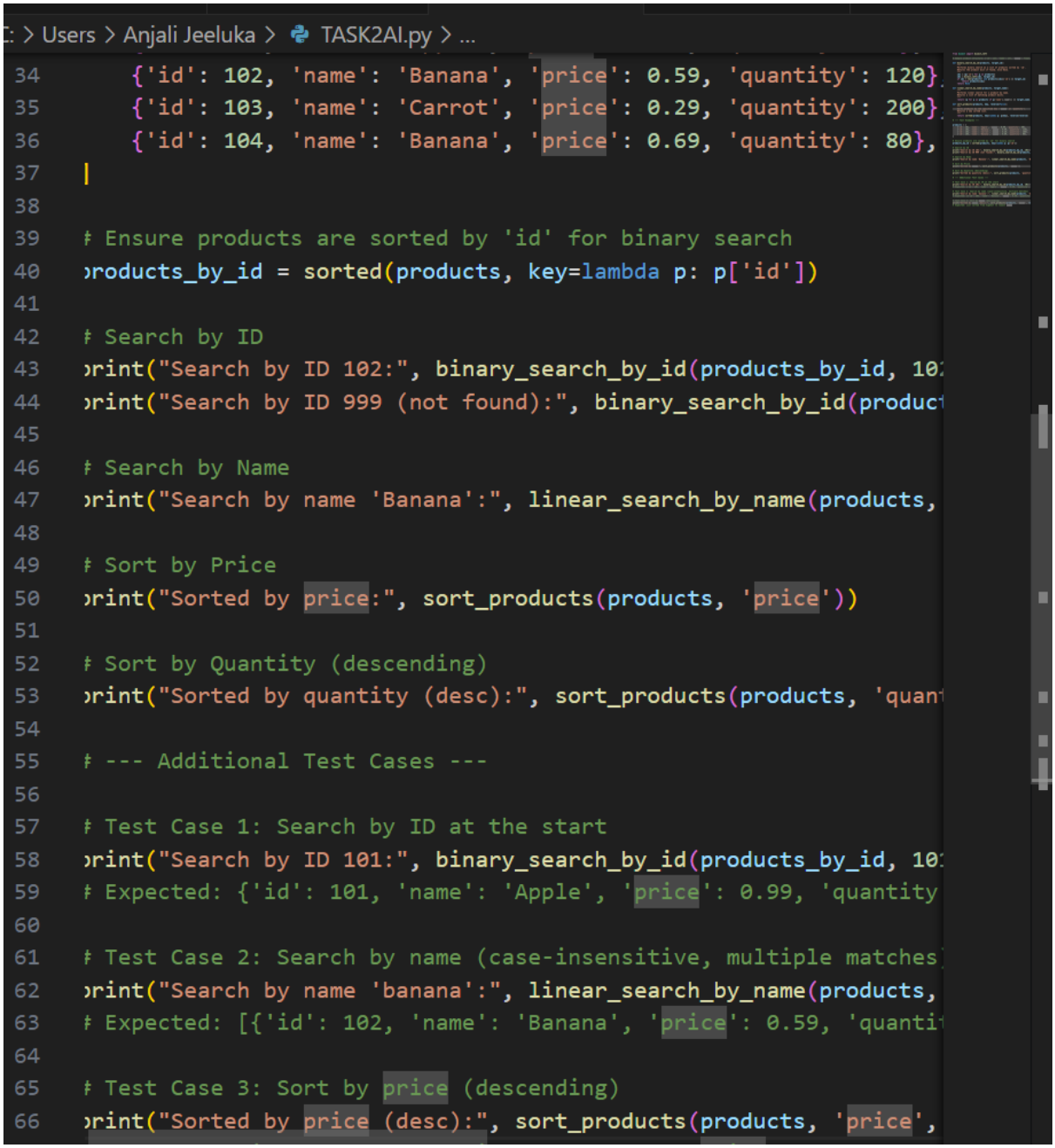
Python functions for searching (by ID, by name) and sorting (by price, by quantity).

Test examples showing how the functions work."

CODE GENERATED :

C: > Users > Anjali Jeeluka > TASK2AI.py > ...

```python
from bisect import bisect_left

# Sample product structure: {'id': int, 'name': str, 'price': flo

def binary_search_by_id(products, target_id):
    """
    Performs binary search on a list of products sorted by 'id'.
    Returns the product dict if found, else None.
    """
    ids = [p['id'] for p in products]
    idx = bisect_left(ids, target_id)
    if idx < len(products) and products[idx]['id'] == target_id:
        return products[idx]
    return None

def linear_search_by_name(products, target_name):
    """
    Performs linear search for a product by name.
    Returns a list of matching product dicts.
    """
    return [p for p in products if p['name'].lower() == target_na

def sort_products(products, key, reverse=False):
    """
    Sorts products by the specified key ('price' or 'quantity').
    Returns a new sorted list.
    """
    return sorted(products, key=lambda p: p[key], reverse=reverse

# --- Test Examples ---

products = [
```



```
C: > Users > Anjali Jeeluka > TASK2AI.py > ...
34      {'id': 102, 'name': 'Banana', 'price': 0.59, 'quantity': 120},
35      {'id': 103, 'name': 'Carrot', 'price': 0.29, 'quantity': 200},
36      {'id': 104, 'name': 'Banana', 'price': 0.69, 'quantity': 80},
37  ]
38
39  # Ensure products are sorted by 'id' for binary search
40  products_by_id = sorted(products, key=lambda p: p['id'])
41
42  # Search by ID
43  print("Search by ID 102:", binary_search_by_id(products_by_id, 10
44  print("Search by ID 999 (not found):", binary_search_by_id(produc
45
46  # Search by Name
47  print("Search by name 'Banana':", linear_search_by_name(products,
48
49  # Sort by Price
50  print("Sorted by price:", sort_products(products, 'price'))
51
52  # Sort by Quantity (descending)
53  print("Sorted by quantity (desc):", sort_products(products, 'quan
54
55  # --- Additional Test Cases ---
56
57  # Test Case 1: Search by ID at the start
58  print("Search by ID 101:", binary_search_by_id(products_by_id, 10
59  # Expected: {'id': 101, 'name': 'Apple', 'price': 0.99, 'quantity
60
61  # Test Case 2: Search by name (case-insensitive, multiple matches
62  print("Search by name 'banana':", linear_search_by_name(products,
63  # Expected: [{'id': 102, 'name': 'Banana', 'price': 0.59, 'quanti
64
65  # Test Case 3: Sort by price (descending)
66  print("Sorted by price (desc):", sort_products(products, 'price',
```

```
64
65  # Test Case 3: Sort by price (descending)
66  print("Sorted by price (desc):", sort_products(products, 'price',
67  # Expected: List sorted from highest to lowest price
```

OUTPUT :

```
PS C:\Users\Anjali Jeeluka\.vscode> & "C:/Users/Anjali Jeeluka/an
aconda3/python.exe" "c:/Users/Anjali Jeeluka/TASK2AI.py"
Search by ID 102: {'id': 102, 'name': 'Banana', 'price': 0.59, 'q
uantity': 120}
Search by ID 999 (not found): None
Search by name 'Banana': [{'id': 102, 'name': 'Banana', 'price':
0.59, 'quantity': 120}, {'id': 104, 'name': 'Banana', 'price': 0.
69, 'quantity': 80}]
Sorted by price: [{'id': 103, 'name': 'Carrot', 'price': 0.29, 'q
uantity': 200}, {'id': 102, 'name': 'Banana', 'price': 0.59, 'qua
ntity': 120}, {'id': 104, 'name': 'Banana', 'price': 0.69, 'quant
ity': 80}, {'id': 101, 'name': 'Apple', 'price': 0.99, 'quantity'
: 50}]
Sorted by quantity (desc): [{'id': 103, 'name': 'Carrot', 'price'
: 0.29, 'quantity': 200}, {'id': 102, 'name': 'Banana', 'price':
0.59, 'quantity': 120}, {'id': 104, 'name': 'Banana', 'price': 0.
69, 'quantity': 80}, {'id': 101, 'name': 'Apple', 'price': 0.99,
'quantity': 50}]
Search by ID 101: {'id': 101, 'name': 'Apple', 'price': 0.99, 'qu
antity': 50}
Search by name 'banana': [{'id': 102, 'name': 'Banana', 'price':
0.59, 'quantity': 120}, {'id': 104, 'name': 'Banana', 'price': 0.
69, 'quantity': 80}]
Sorted by price (desc): [{'id': 101, 'name': 'Apple', 'price': 0.
99, 'quantity': 50}, {'id': 104, 'name': 'Banana', 'price': 0.69,
 'quantity': 80}, {'id': 102, 'name': 'Banana', 'price': 0.59, 'q
uantity': 120}, {'id': 103, 'name': 'Carrot', 'price': 0.29, 'qua
ntity': 200}]
PS C:\Users\Anjali Jeeluka\.vscode>
```

OBSERVATION :

- **Functionality:**
  The code provides efficient search and sort utilities for a product inventory system, including:
  - Binary search by product ID (requires sorted input).
  - Linear search by product name (case-insensitive).
  - Sorting by price or quantity, with support for ascending/descending order.

- **Test Coverage:**
  Multiple test cases are included, covering:
  - Searching for existing and non-existing IDs.
  - Searching by name with different cases and multiple matches.
  - Sorting by price and quantity in both ascending and descending order.
- **Code Quality:**
  - Functions are well-named and include clear docstrings.
  - The code follows PEP 8 guidelines for formatting and readability.
  - The use of bisect_left for binary search is efficient and appropriate for large, sorted datasets.
  - The linear search by name is case-insensitive, improving usability.
  - Sorting uses Python's built-in sorted() for efficiency and stability.
- **Edge Cases:**
  - Handles multiple products with the same name.
  - Handles searching for IDs that do not exist.
  - Sorting works for both increasing and decreasing order.
- **Potential Improvements:**
  - For very large datasets with frequent lookups, consider using a dictionary for O(1) ID searches.
  - For name searches, if names are unique and sorted, binary search could be used for further optimization.
  - Input validation and error handling could be added for robustness.
- **Overall:**
  The code is robust, efficient, and well-structured for typical inventory search and sort operations in a retail context.